

Sistemi informativi su Web

Spring Boot: Conversione e Validazione dei Dati di Input aa 2024-2025



Paolo Merialdo
Università degli Studi Roma Tre



This work is licensed under a Creative Commons Attribution-Noncommercial-Share Alike 3.0 United States
See <http://creativecommons.org/licenses/by-nc-sa/3.0/us/> for details

Introduzione

- La gestione dei dati raccolti attraverso le form è una operazione delicata
- La costruzione di una form HTML è semplice, ma controllare se ogni campo ha un valore valido e coerente, e conseguente segnalare eventuali errori all'utente introduce numerose complicazioni
- La validazione può avvenire
 - Lato client: il browser verifica la correttezza dei dati raccolti attraverso una form prima di inviare la richiesta HTTP al server
 - Lato server: il server verifica la correttezza di tutti i dati che arrivano da una richiesta HTTP

Validazione lato client

- HTML5 ha introdotto nuovi meccanismi per la gestione delle form:
 - Tipi semantici per l'elemento `<input>` e la validazione dei vincoli per facilitare il lavoro di verifica del contenuto del lato client.
 - I vincoli di base possono essere verificati facilmente, i vincoli più articolati possono essere implementati tramite JavaScript
- Per una panoramica della validazione lato client con HTML5 fare riferimento a MDN

https://developer.mozilla.org/en-US/docs/Web/HTML/Constraint_validation

Validazione lato client e lato server

- La convalida dei dati lato client HTML non elimina la necessità di effettuare una convalida sul lato server
- Anche se ci si può aspettare un numero molto inferiore di richieste con dati non validi, è comunque possibile ricevere dati non validi, ad esempio da persone malintenzionate
- Pertanto, è necessario validare **sempre** i dati anche lato server, in modo coerente con quanto fatto lato client

Validazione lato server: soluzione naïve

- Possiamo costruire una soluzione lato server mettendo nel controller il codice per effettuare le verifiche
 - Per ogni campo verifichiamo la correttezza (ad es., titolo del film diverso da stringa vuota, anno del film un intero compreso tra 1900 e l'anno corrente)
 - In caso di errori, generiamo nuovamente la form, con i campi precompilati ed un opportuno messaggio di errore

Esempio soluzione naïve #1

```
@PostMapping("/movie")
public String newMovie(@ModelAttribute("movie") Movie movie) {
    if (movie.getTitle()==null || movie.getTitle().equals("")) {
        return "formNewMovie.html";
    }
    else
        this.movieService.save(movie);
    return "redirect:movie/"+movie.getId();
}
```

In questa soluzione, torniamo alla form (con i campi prempilati), ma l'utente non capisce cosa sia successo. Serve un messaggio di errore

Esempio soluzione naïve #2

```
@PostMapping("/movie")
public String newMovie(@ModelAttribute("movie") Movie movie, Model model) {
    if (movie.getTitle()==null || movie.getTitle().equals("")) {
        model.addAttribute("msgErrTitle", "Campo obbligatorio");
        return "formNewMovie.html";
    }
    else
        this.movieService.save(movie);
    return "redirect:movie/"+movie.getId();
}
```

In questa soluzione, passiamo alla form un messaggio di errore che può essere mostrato all'utente (vedi template prox slide)

Esempio soluzione naïve #2

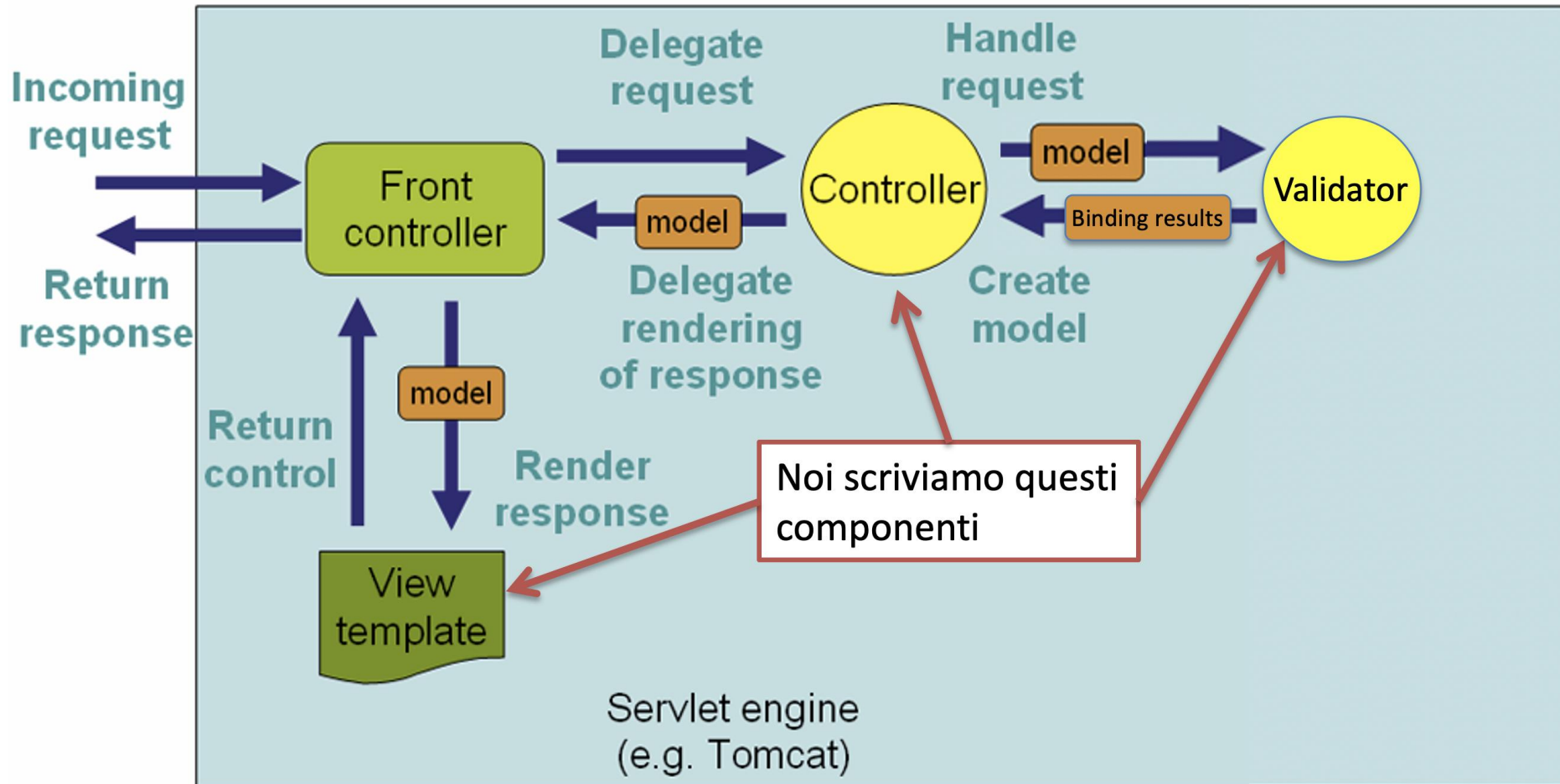
formNewMovie.html

```
</html>
...
<h1>Aggiungi un film</h1>
<form th:action="@{/movie}" method="POST" th:object="${movie}">
  <div>
    <label for="titolo">Titolo:</label>
    <span>
      <input required type="text" th:field="${movie.title}" id="titolo"/>
      <span th:text="${msgErrTitle}">messaggio errore</span>
    </span>
  </div>
  ...
</form>
...
</html>
```

Soluzione naïve

- La soluzione che abbiamo visto è semplice concettualmente
- Ma scrivere il codice per validare tutti i campi è estremamente costoso
- Spring Boot offre una soluzione che semplifica drasticamente questo compito

Spring Boot: Architettura MVC



Spring Boot: Controller

- Spring Boot semplifica la definizione dei Controller MVC
 - Ogni controller MVC è definito come una normale classe Java, annotata `@Controller`
 - E' possibile mappare ogni metodo del controller con un URL tramite le annotazioni `@GetMapping` e `@PostMapping` (rispettivamente per richieste HTTP GET e POST).
 - I parametri HTTP (che sono trasmessi sempre come stringhe di testo) sono passati come normali parametri Java (opportunamente annotati) e convertiti automaticamente nel tipo di destinazione.
Attenzione: la conversione potrebbe non andare a buon fine (ad esempio, il valore di un campo Integer è una stringa alfabetica)
 - Ogni metodo ritorna una stringa che rappresenta il nome della vista successiva
 - I dati da passare alla vista vengono aggiunti come attributi di un oggetto **Model**

Spring Boot: Validazione

- I vincoli sui dati di input sono proprietà delle entità
- Il tipo dei dati (Integer, String, Double) rappresenta un vincolo implicito
- Ma è possibile specificare in maniera dichiarativa ulteriori vincoli di validità dei dati direttamente sugli attributi delle entità attraverso opportune annotazioni (del package `jakarta.validation.constraints`)
- Per usare queste librerie, quando creiamo il progetto Spring Boot dobbiamo importare anche le librerie **Validation**
- Per un elenco completo delle annotazioni vedi:
<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/constraints/package-summary.html>

Vincoli Validazione: esempio

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.Max;

@Entity
public class Movie {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @NotBlank
    private String title;

    @NotNull
    @Min(1900)
    @Max(2023)
    private Integer year;

    ...
}
```

NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

Spring Boot: Validazione

- Violazioni dei vincoli possono essere gestite automaticamente dai nostri Controller MVC
- Abbiamo visto che un metodo del controller può prendere come parametro una entità
- Se i parametri HTTP hanno lo stesso nome delle variabili dell'entità, il controller (facendo ampio uso della riflessione) prende i valori dei parametri HTTP per creare un oggetto istanza dell'entità, preoccupandosi di **convertire** e **validare** i dati
 - Questa operazione si chiama "binding"
- Eventuali errori (di **conversione** o di **validazione**) rilevati durante il *binding* vengono riportati in un opportuno oggetto

Validatori: esempio

@Controller

```
public class MovieController {
```

```
    @Autowired MovieService movieService;
```

```
    @PostMapping("/movie")
```

```
    public String newMovie(@Valid @ModelAttribute("movie") Movie movie,  
                           BindingResult bindingResult, Model model) {
```

```
        if (bindingResult.hasErrors()) { // sono emersi errori nel binding
```

```
            return "formNewMovie.html";
```

```
        } else { // NON sono emersi errori nel binding
```

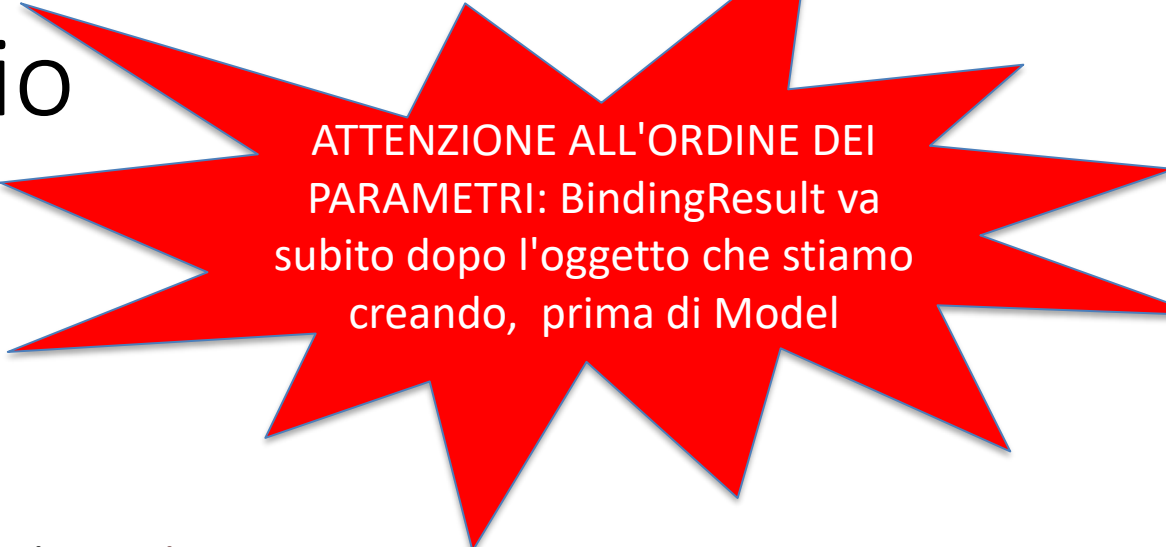
```
            this.movieService.save(movie);
```

```
            model.addAttribute("movie", movie);
```

```
            return "redirect:movie/"+movie.getId();
```

```
        }
```

```
    }
```



ATTENZIONE ALL'ORDINE DEI
PARAMETRI: BindingResult va
subito dopo l'oggetto che stiamo
creando, prima di Model

NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

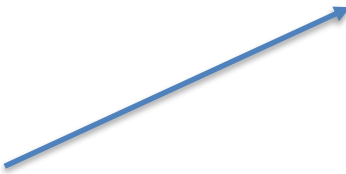
Spring Boot: Validazione

- Ad ogni errore di validazione è associato un messaggio di errore
 - Questi messaggi possono essere usati nelle viste, per interagire con l'utente
- Ogni errore ha un messaggio di default, ma è possibile personalizzare i messaggi

Spring Boot: Configurazione messaggi

```
#=====
# = Datasource configuration
#=====
spring.jpa.database=POSTGRESQL
....
#=====
# = Webserver configuration
#=====
server.port= 8080
...
#=====
# = Messages configuration
#=====
spring.messages.basename=messages/messages
spring.messages.encoding=ISO-8859-1
```

I file con I messsaggi si trovano nella directory **messages** (dentro **resources**) ed il suffisso del nome del file e' **messages**.



NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

Personalizzazione Messaggi Errore

- I messaggi di errore personalizzati sono scritti in un opportuno file di configurazione
 - Il path del file è specificato in `application.properties`

```
#=====
# = Messages configuration
#=====
spring.messages.basename=messages/messages
spring.messages.encoding=ISO-8859-1
```

Aggiungiamo nel file `application.properties`

- Ad ogni chiave (codice di errore) associamo un messaggio

```
movie.duplicate=Questo film esiste già
NotBlank=Campo obbligatorio
NotNull=Campo obbligatorio
Min.year=Deve essere un numero maggiore di 1900
Max.year=Deve essere un numero minore di 2023
NotBlank.movie.title=Il titolo è obbligatorio
NotBlank.movie.year=Campo obbligatorio
typeMismatch.java.lang.Integer=Deve essere un numero
typeMismatch=Formato non valido
```

Contenuto del file `messages/messages.properties`

Chiavi dei messaggi di errore

- `NotBlank=`[Campo obbligatorio](#)
Si applica a tutti i campi in cui si verifica una violazione del vincolo NotBlank
- `NotBlank.movie=`[Campo obbligatorio](#)
Si applica a tutti i campi dell'entità Movie in cui si verifica una violazione del vincolo NotBlank
- `NotBlank.movie.title=`[Il titolo è obbligatorio](#)
Si applica al campo title dell'entità Movie in caso di violazione del vincolo NotBlank
- `typeMismatch.java.lang.Integer=`[Deve essere un numero](#)
Si applica una violazione di tipo Integer
- `typeMismatch=`[Formato non valido](#)
Si applica una generica violazione di tipo

In caso di conflitti, viene usata sempre la chiave più specifica

Messaggi di errore in Thymeleaf

- Per la gestione degli errori, Thymeleaf permette verificare se su qualche campo c'è un errore e di accedere ai messaggi di errore di ogni campo

```
<span>Titolo:</span>
```

```
<span><input type="text" th:field="*{title}" /></span>
```

```
<span th:if="$#{#fields.hasErrors('title')}" th:errors="*{title}"></span>
```

- `#fields.hasErrors('...')`: funzione che riceve come parametro un campo, e ritorna un booleano che riporta se c'è stato un qualche errore di validazione per quel campo (in pratica accede alla variabile `BindingResults`).
- `th:errors`, uno speciale attributo che costruisce una lista di tutti gli errori del campo selezionato, separati da un tag `<br />` (*break row*, va a capo)

Thymeleaf: form

```
<!DOCTYPE html>
<html>
<head>
  <title>New Movie</title>
  <link rel="stylesheet" href="/stile.css?version=3" />
</head>

<body>
  <div><a href="/"></a></div>
  <h1>Aggiungi un film</h1>
  <form th:action="@{/movie}" method="POST" th:object="${movie}">
    <div th:if="${#fields.hasGlobalErrors()}">
      <p th:each="err : ${#fields.globalErrors()}" th:text="${err}">...</p>
    </div>
    <div>
      <label>Titolo:</label> <span><input type="text" th:field="${movie.title}" /></span>
      <span th:if="${#fields.hasErrors('title')}" th:errors="*{title}"></span>
    </div>
    <div>
      <label>Anno:</label> <span><input type="text" th:field="${movie.year}" /></span>
      <span th:if="${#fields.hasErrors('year')}" th:errors="*{year}"></span>
    </div>
    <div>
      <label>URL immagine:</label> <span><input type="text" th:field="${movie.urlImage}" /></span>
    </div>
    <div>
      <span><button type="submit">Conferma</button></span>
    </div>
  </form>
</body>
</html>
```

NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

```
<form th:action="@{/movie}" method="POST" th:object="${movie}">
  <div th:if="${#fields.hasGlobalErrors()}">
    <p th:each="err : ${#fields.globalErrors()}" th:text="${err}">...</p>
  </div>
  <div>
    <label>Titolo:</label> <span><input type="text" th:field="${movie.title}" /></span>
    <span th:if="${#fields.hasErrors('title')}" th:errors="*{title}"></span>
  </div>
  <div>
    <label>Anno:</label> <span><input type="text" th:field="${movie.year}" /></span>
    <span th:if="${#fields.hasErrors('year')}" th:errors="*{year}"></span>
  </div>
  <div>
    <label>URL immagine:</label>
    <span><input type="text" th:field="${movie.urlImage}" /></span>
  </div>
  <div>
    <span><button type="submit">Conferma</button></span>
  </div>
</form>
```

NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

Spring Boot: Validazione Custom

- La validazione considera i vincoli espressi nell'entità
- In alcuni casi però potrebbe essere necessario specificare vincoli particolari, non esprimibili con le annotazioni della libreria
- Per ovviare abbiamo due strade
 - Definiamo una nostra annotazione (ha senso se la riusiamo in altri contesti)
 - Completiamo la validazione con una classe opportuna
- Noi vedremo la seconda soluzione

Spring Boot: Validazione Custom

- Creiamo una classe che implementa l'interfaccia `org.springframework.validation.Validator` nella quale implementiamo il metodo `validate(Object, Errors)`
- Questo metodo prende due parametri:
 - Il primo è l'oggetto da validare
 - Il secondo è un oggetto che riporta l'esito della validazione ed eventuali errori riscontrati (**Errors** è un supertipo di **BindingResults**)

Validazione Custom: Esempio

- Supponiamo che non vogliamo inserire film che sono già stati memorizzate nel nostro sistema
- Scriviamo una classe `MovieValidator` il cui metodo `validate()` implementa questa verifica
- Quindi usiamo questo validatore nel controllo

Validatori: esempio

@Component

```
public class MovieValidator implements Validator {
```

```
@Autowired
```

```
private MovieService movieService;
```

```
@Override
```

```
public void validate(Object o, Errors errors) {
```

```
    Movie movie = (Movie)o;
```

```
    if (movie.getTitle()!=null && movie.getYear()!=null
```

```
        && movieService.existsByTitleAndYear(movie.getTitle(), movie.getYear())) {
```

```
        errors.reject("movie.duplicate");
```

```
    }
```

```
}
```

Specifica che c'e' stato un errore nella
validazione e registra un codice di errore
(la stringa "movie.duplicate")



```
@Override
```

```
public boolean supports(Class<?> aClass) {
```

```
    return Movie.class.equals(aClass);
```

```
}
```

```
}
```

NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

Validatori custom: uso nel controller

@Controller

```
public class MovieController {  
    @Autowired MovieService movieService;  
    @Autowired MovieValidator movieValidator;  
  
    @PostMapping("/movie")  
    public String newMovie(@Valid @ModelAttribute("movie") Movie movie,  
                           BindingResult bindingResult, Model model) {  
        this.movieValidator.validate(movie, bindingResult);  
        if (bindingResult.hasErrors()) { // sono emersi errori nel binding  
            return "formNewMovie.html";  
        } else { // NON sono emersi errori nel binding  
            this.movieService.save(movie);  
            model.addAttribute("movie", movie);  
            return "redirect:movie/"+movie.getId();  
        }  
    }  
}
```

NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

Come verificiamo la validazione lato server?

Togliamo la validazione lato client sul template HTML5

Message expression: esempio

```
<!DOCTYPE html>
<html>
<head>
  <title>Film</title>
</head>
<body>
  <h4 th:text="#{movie.header}">Dettagli film</h4>
  <div th:object=${movie}>
    <span th:text="*{title}">Il buio oltre la siepe</span>
    <span th:text="*{year}">1962</span>
  </div>
</div>
</body>
</html>
```

Questa è una chiave: il valore è riportato nei file in cui memorizziamo i messaggi Internazionalizzati (es. messages_it.properties).

In questo modo, possiamo internazionalizzare tutto il sito

NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

Thymeleaf: espressioni

$\${...}$ - *Variable expressions*: contengono variabili che sono nel modello

$*{...}$ - *Selection expressions*: fanno riferimento ai campi di una variabile definita in uno scope esterno

$\#{...}$ - *Message (i18n) expressions*: fanno riferimento a messaggi di testo scritti nei file di messaggi (predisposti all'internazionalizzazione)

$@{...}$ - *Link (URL) expressions*: espressioni per costruire riferimenti ipertestuali

$\sim{...}$ - *Fragment expressions*: espressioni per includere frammenti di template (vedi oltre)

Per dettagli: <https://www.thymeleaf.org/doc/articles/standarddialect5minutes.html>

Selection expression: esempio

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
...
```

```
</head>
```

```
<body>
```

```
<div>
```

```
<h4>Dettagli fil</h4>
```

```
<div th:object=${movie}>
```

```
<span th:text="{title}">Il buio oltre la siepe</span>
```

```
<span th:text="{year}">1962</span>
```

```
...
```

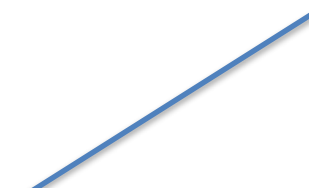
```
</div>
```

```
</div>
```

```
...
```

```
</html>
```

Facciamo riferimento all'oggetto definito
nello scope esterno (persona)



NON FARE COPIA E INCOLLA DEL CODICE DALLE SLIDE:
POWERPOINT INTRODUCE CARATTERI BIANCHI CHE SPORCANO IL CODICE

Selection expression: esempio

```
<div th:object=${movie}>  
  <span th:text="{title}">Il buio oltre la siepe</span>  
  <span th:text="{year}">1962</span>  
</div>
```

Equivalente a:

```
<div>  
  <span th:text="{movie.title}">Il buio oltre la siepe</span>  
  <span th:text="{movie.year}">1962</span>  
</div>
```

Molto comodo quando ci sono tanti campi

Conversione e validazione lato server: sintesi

- Annotiamo i campi delle entità per specificare vincoli sui dati
- Definiamo eventuali vincoli custom attraverso una classe che implementa l'interfaccia Validator
- Chiediamo al Controller di validare i dati di input
 - marcandoli nella segnatura del metodo con l'annotazione @Valid e passandogli il parametro BindingResults
 - Nel corpo del controller verifichiamo che il binding dei dati non abbia prodotto errori (di conversione e/o di validazione)
- Nei template mostriamo gli eventuali errori

Un (Piccolo) Problema Frequente

- In Spring Boot, la White Label Error Page è la pagina di errore predefinita che viene mostrata al browser quando l'applicazione genera un errore HTTP (come un 404, 500, ecc.) e non si è definita una pagina di errore personalizzata.
- Quando compare?
 - Se un URL non è mappato (errore 404)
 - Se un controller genera un'eccezione (errore 500)
 - Se c'è un problema interno all'applicazione
- Chiedere ad un LLM come disabilitarla e come creare una propria Error Page